

CSC 345 Project: 2

Full Name: Nick Viola & Sofia Hall

Section: 02

0. Weekly Discussion Note

N/A

1. Input Handling & Formatting

1.1. Source Code

```
151  /* function to read in board from input.txt */
152  void readBoard() {
153      FILE *fptr = fopen("input.txt", "r");
154
155      /* enters input.txt into array */
156      for (int i = 0; i < 9; i++) {
157          for (int j = 0; j < 9; j++) {
158              fscanf(fptr, "%d", &board[i][j]);
159          }
160      }
161      fclose(fptr);
162  }
163
164  /* function to print out board */
165  void printBoard() {
166      printf("BOARD STATE IN input.txt:\n"); /* header */
167
168      for (int i = 0; i < 9; i++) {
169          for (int j = 0; j < 9; j++) {
170              printf("%d ", board[i][j]);
171          }
172          printf("\n"); /* newline after 9 numbers */
173      }
174  }
```

Figure 1. Source code for input handling and output format.

1.2. Output

```
tcnj@tcnj-osc:~/Project_2_CSC345$ ./main 1
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.0005 seconds)
```

Figure 2. Terminal output of sudoku board and validation.

Figure 2 shows the program reading the Sudoku board taken from *input.txt* and printing it out in the terminal. This program displays the board and determines whether it has a valid solution and how long it takes to solve.

2. Multithreading Implementation

2.1. Source Code – Method 1

In method 1, 11 threads are distributed. 1 thread is used for all the rows, 1 thread is used for all the columns, and 9 threads are used for the 3 by 3 sub grids within the puzzle. The functions to check the rows, columns, and sub grids can be seen below.

```
34 /* checks each row if it contains all digits 1-9 */
35 void *checkRows1(void *param) {
36     parameters *data = (parameters*) param;
37     int idx = data->index;
38
39     int seen[10];
40
41     for (int i = 0; i < 9; i++) {
42         for (int j = 0; j < 10; j++) { seen[j] = 0; }
43
44         for (int j = 0; j < 9; j++) {
45             int val = board[i][j];
46             if (val < 1 || val > 9 || seen[val] != 0) {
47                 results[idx] = 0; /* returns 0 if duplicates */
48                 pthread_exit(NULL);
49             }
50             seen[val] = 1;
51         }
52     }
53     results[idx] = 1; /* returns 1 if all digits 1-9 are seen */
54     pthread_exit(NULL);
55 }
```

Figure 3. Method 1 check rows function

```

57  /* checks each col if it contains all digits 1-9 */
58  void *checkCols1(void *param) {
59      parameters *data = (parameters*) param;
60      int idx = data->index;
61
62      int seen[10];
63
64      for (int i = 0; i < 9; i++) {
65          for (int j = 0; j < 10; j++) { seen[j] = 0; }
66
67          for (int j = 0; j < 9; j++) {
68              int val = board[j][i];
69              if (val < 1 || val > 9 || seen[val] != 0) {
70                  results[idx] = 0; /* returns 0 if duplicates */
71                  pthread_exit(NULL);
72              }
73              seen[val] = 1;
74          }
75      }
76      results[idx] = 1; /* returns 1 if all digits 1-9 are seen */
77      pthread_exit(NULL);
78  }

```

Figure 4. Method 1 check columns function

```

80  /* checks 3 by 3 if it contains all digits 1-9 */
81  void *checkSquare(void *param) { /* used by method 1 and 2 */
82      parameters *data = (parameters *) param;
83      int row = data->row;
84      int col = data->column;
85      int idx = data->index;
86
87      int seen[10];
88
89      for (int i = 0; i < 10; i++) { seen[i] = 0; }
90
91      for (int i = row; i < row + 3; i++) {
92          for (int j = col; j < col + 3; j++) {
93              int val = board[i][j];
94              if (val < 1 || val > 9 || seen[val] != 0) {
95                  results[idx] = 0; /* returns 0 if duplicates */
96                  pthread_exit(NULL);
97              }
98              seen[val] = 1;
99          }
100      }
101      results[idx] = 1; /* returns 1 if 1-9 is in square */
102      pthread_exit(NULL);
103  }

```

Figure 5. Check sub grid function (same for both methods)

Once these three functions are created, the actual threads can be created in the main function when “1” is called as the argument.

```

190 if (mode == 1) { /* Method 1: 11 threads */
191     /* thread 1: all rows, thread 2: all cols, threads 3-11: subgrids */
192
193     pthread_t threads[11];
194     parameters* data[11];
195
196     for (int i = 0; i < 11; i++) {
197         data[i] = (parameters *) malloc(sizeof(parameters));
198         results[i] = 0;
199     }
200
201     /* row thread */
202     data[0]->row = 0;
203     data[0]->column = 0;
204     data[0]->index = 1;
205     pthread_create(&threads[0], NULL, checkRows1, data[0]);
206
207     /* col thread */
208     data[1]->row = 0;
209     data[1]->column = 0;
210     data[1]->index = 0;
211     pthread_create(&threads[1], NULL, checkCols1, data[1]);
212
213     /* subgrids threads */
214     for (int i = 2; i < 11; i++) {
215         data[i]->row = rows[i - 2];
216         data[i]->column = cols[i - 2];
217         data[i]->index = i;
218         pthread_create(&threads[i], NULL, checkSquare, data[i]);
219     }
220
221     for (int i = 0; i < 11; i++) {
222         pthread_join(threads[i], NULL);
223     }
224
225     int valid = 1;
226     for (int i = 0; i < 11; i++) {
227         if (results[i] == 0) {
228             valid = 0;
229             break;
230         }
231     }
232
233     clock_gettime(CLOCK_MONOTONIC, &end);
234     double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
235
236     /* printing to the terminal */
237     if (valid) { printf("SOLUTION: YES (%.4f seconds)\n", elapsed); }
238     else { printf("SOLUTION: NO\n"); }
239
240     /* freeing the memory */
241     for (int i = 0; i < 11; i++) { free(data[i]); }
242
243 }

```

Figure 6. Source code for thread creation for 11 threads.

2.2. Source Code – Method 2

Similarly to method one, functions to check rows and columns need to be created. Since this method calls for 27 threads, each row or column will create its own thread. This causes the necessity for new check rows and columns functions to be made. However, the implementation of the sub grid checker remains the same as it requires the same number of threads. The 27 threads are distributed to have 9 for the rows, 9 for the columns, and 9 for the sub grids.

```
107  /* checks one row if it contains all digits 1-9 */
108  void *checkRows2(void *param) {
109      parameters *data = (parameters*) param;
110      int row = data->row;
111      int idx = data->index;
112
113      int seen[10];
114
115      for (int j = 0; j < 10; j++) { seen[j] = 0; }
116
117      for (int j = 0; j < 9; j++) {
118          int val = board[row][j]; /* checking only 1 row */
119          if (val < 1 || val > 9 || seen[val] != 0) {
120              results[idx] = 0; /* returns 0 if duplicates */
121              pthread_exit(NULL);
122          }
123          seen[val] = 1;
124      }
125      results[idx] = 1; /* returns 1 if all digits 1-9 are seen */
126      pthread_exit(NULL);
127  }
```

Figure 7. Method 2 check rows function

```
129  /* checks one col if it contains all digits 1-9 */
130  void *checkCols2(void *param) {
131      parameters *data = (parameters*) param;
132      int col = data->column;
133      int idx = data->index;
134
135      int seen[10];
136
137      for (int j = 0; j < 10; j++) { seen[j] = 0; }
138
139      for (int i = 0; i < 9; i++) {
140          int val = board[i][col]; /* checking only 1 row */
141          if (val < 1 || val > 9 || seen[val] != 0) {
142              results[idx] = 0; /* returns 0 if duplicates */
143              pthread_exit(NULL);
144          }
145          seen[val] = 1;
146      }
147      results[idx] = 1; /* returns 1 if all digits 1-9 are seen */
148      pthread_exit(NULL);
149  }
```

Figure 8. Method 2 check columns function

The main function for the second method is very similar to the first method, except for when the threads for the rows and columns are created; a for loop is used to pass all the row and column

threads. This is due to 9 threads for rows and columns, versus only 1 for all rows and 1 for all columns of the first method. The execution of the sub grid threads remains the same.

```
177 int main(int argc, char *argv[]) {
245     else if (mode == 2) { /* Method 2: 27 threads */
246         /* threads 1-9: each row, threads 10-18: each cols, threads 19-27: subgrids */
247
248         pthread_t threads[27];
249         parameters* data[27];
250
251         for (int i = 0; i < 27; i++) {
252             data[i] = (parameters *) malloc(sizeof(parameters));
253             results[i] = 0;
254         }
255
256         /* row threads */
257         for (int i = 0; i < 9; i++) {
258             data[i]->row = i;
259             data[i]->column = 0;
260             data[i]->index = i;
261             pthread_create(&threads[i], NULL, checkRows2, data[i]);
262         }
263
264         /* col threads */
265         for (int i = 9; i < 18; i++) {
266             data[i]->row = 0;
267             data[i]->column = i - 9;
268             data[i]->index = i;
269             pthread_create(&threads[i], NULL, checkRows2, data[i]);
270         }
271
272         /* subgrids threads */
273         for (int i = 18; i < 27; i++) {
274             data[i]->row = rows[i - 18];
275             data[i]->column = cols[i - 18];
276             data[i]->index = i;
277             pthread_create(&threads[i], NULL, checkSquare, data[i]);
278         }
279
280         for (int i = 0; i < 27; i++) {
281             pthread_join(threads[i], NULL);
282         }
283
284         int valid = 1;
285         for (int i = 0; i < 27; i++) {
286             if (results[i] == 0) {
287                 valid = 0;
288                 break;
289             }
290         }
291
292         clock_gettime(CLOCK_MONOTONIC, &end);
293         double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
294
295         /* printing to the terminal */
296         if (valid) { printf("SOLUTION: YES (%.4f seconds)\n", elapsed); }
297         else { printf("SOLUTION: NO\n"); }
298
299         /* freeing the memory */
300         for (int i = 0; i < 27; i++) { free(data[i]); }
301     }
302
303     else { printf("Input error. Please choose between 1 or 2\n"); }
304
305     return 0;
306 }
307 }
```

Figure 9. Source code for thread creation for 27 threads.

```

22  /* structure for passing data to threads */
23  typedef struct {
24      int row;
25      int column;
26      int index;
27  } parameters;

```

Figure 10. *Struct* parameters.

2.3. Output

```

tcnj@tcnj-osc:~/Project_2_CSC345$ ./main 2
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.0011 seconds)

```

Figure 11. Terminal output showing execution using 27 threads.

Figure 11 shows the execution of the program using 27 threads. Using individual threads, each row, column, and sub grid is validated which increases parallelism levels.

3. Statistical Experiment

3.1. Results

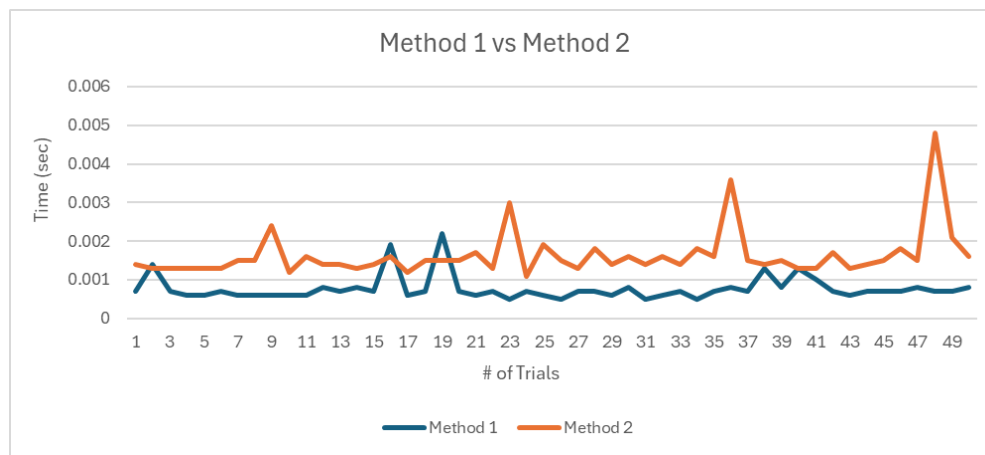
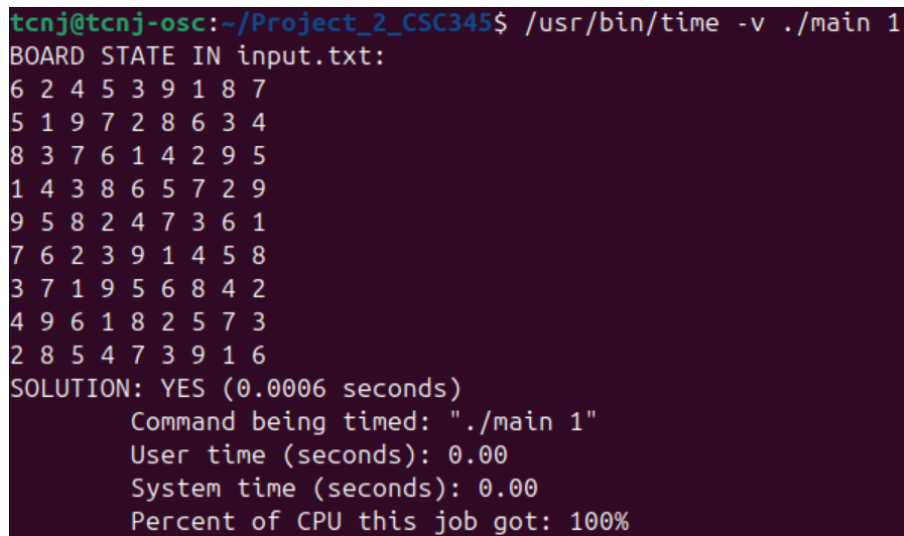


Figure 12. Execution time results for 50 trials.

The plotted values were calculated by running the program 50 times for each threading method and then taking an average of execution time. This figure represents the execution time of the two threading methods with the X-axis showing the run number and the Y-axis representing the execution time which helps in measuring the cost of the two threading methods. The following plot shows the execution time of Method 1 and Method 2, respectively. Method 2 is generally more efficient due to its level of parallelism. Method 2 is faster than Method 1 on average, which shows that using more threads improves the time consumed for execution of each algorithm. The null hypothesis, which asserts that there is no difference between the two methods, is rejected.

4. Multi-Core Utilization

4.1. Output



```
tcnj@tcnj-osc:~/Project_2_CSC345$ /usr/bin/time -v ./main 1
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.0006 seconds)
    Command being timed: "./main 1"
    User time (seconds): 0.00
    System time (seconds): 0.00
    Percent of CPU this job got: 100%
```

Figure 13. CPU usage during program execution.

The command was used to measure CPU usage, and showed a 100% CPU usage, indicating that the program ran on a single core. This doesn't mean the program can't use multiple cores. The Sudoku simulation is lightweight, completing faster than 0.001 for most trials using the first method. Because it's so short, the LINUX OS scheduler doesn't integrate threads across multiple cores, as the task would already be finished. The program uses pthreads which can run in parallel across multiple cores in larger scale projects. Therefore, no changes to the program are necessary.

5. Makefile

5.1. Source Code

```
1 all:
2     gcc -c main.c
3     gcc -o main main.o -lpthread
4     ./main 1
5     ./main 2
6
7
8 clean:
9     rm main.o
10    rm main
```

Figure 14. Makefile used to compile Project 2 main program.

5.2. Output

```
tcnj@tcnj-osc:~/Desktop/CSC345/Project_2_CSC345$ make clean
rm main.o
rm main
tcnj@tcnj-osc:~/Desktop/CSC345/Project_2_CSC345$ make
gcc -c main.c
gcc -o main main.o -lpthread
./main 1
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.0011 seconds)
./main 2
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.0022 seconds)
```

Figure 15. Terminal output showing successful compilation and cleanup using the Makefile.

The terminal output shows the calls to Makefile and the executions. The *make* command is executed so the project can work in the terminal. At this point, *make clean* is run to delete the compiled programs from the disk.